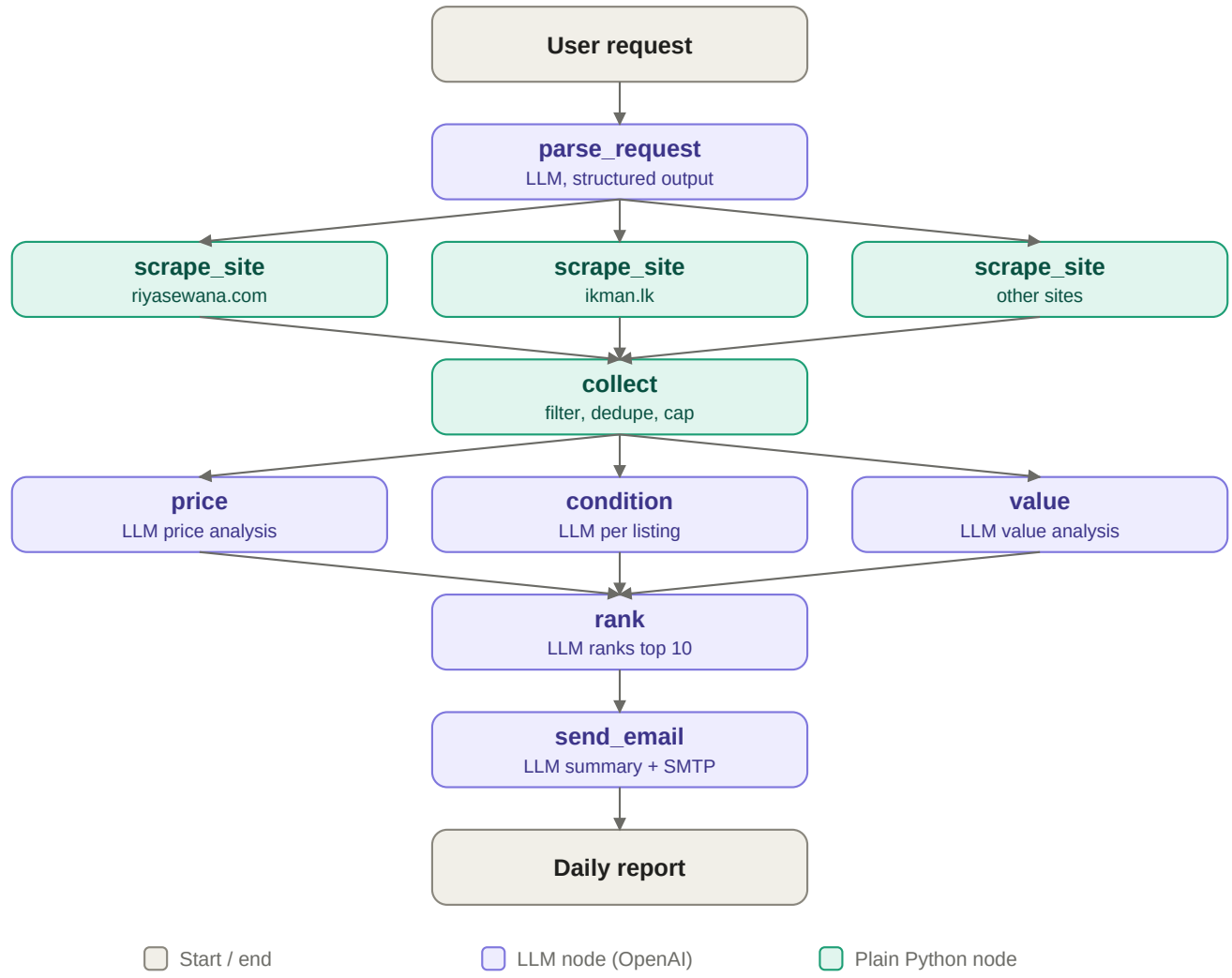
Vehicle Search Agent

A multi-agent system that searches Sri Lankan car sites (riyasewana.com, ikman.lk) every day for a buyer's saved search, analyses price, condition and value, and emails the top 10 cars.

How a search runs

A buyer enters the vehicle they want, their email, the sites and a daily time in the React app. The search is saved and scheduled. At that time each day (or on **Run now**) the agents below run as one LangGraph pipeline:



What each agent does

Agent	Kind	What it does	Output
parse_request	**LLM**	Reads the buyer's free-text request (e.g. *"Lancer 2005-2015 automatic under 8 million"*) and turns it into search criteria. Understands Sri Lankan amounts like '9M' or '90 lakhs'.	make, model, year range, budget (LKR), vehicle type, gear, fuel
scrape_site	**Python**	Runs once per selected site, all in parallel. Builds the site's own search URL from the criteria (`sites.py`) so the site applies the filters, fetches the results page with Tavily Extract, and reads page 2 if there are more results. If the model matches nothing it retries with the make only; sites without a URL builder use Tavily web search.	Car cards: title, price, year, mileage, location, link to the ad

Agent	Kind	What it does	Output
sanitize	**Python**	Part of scraping. Strips menus, images, footers and ads for other cars from the page and keeps only the fields the agents need. Drops spare-part ads; treats mileage of 1,000 km or less as unknown.	Compact product records
collect	**Python**	Merges all sites, drops cars of the wrong make, year or over budget, removes duplicates (the same car listed twice), alternates between sites and caps the list (25 cars) before any LLM call.	Final list, each car with an ID (L1, L2, ...)
price	**LLM**	Compares each asking price with similar cars (model, year, mileage) in the same results.	One-line price comment per car + market summary
condition	**LLM**	Judges each car's condition from the seller's description (mileage, owners, service history, accidents). Cards without a description get a neutral score without an LLM call, to save cost.	Score 1-10, highlights, red flags
value	**LLM**	Scores how well each car matches the buyer and what they get for the money (spec, year, mileage, resale value).	Value score 1-10 + reason
rank	**LLM**	Waits for price, condition and value, weighs them together and picks the best cars. Skips unknown or repeated IDs.	Top 10 with a reason for each
send_email	**LLM**	Writes a short summary of today's best options and what to watch out for, then emails it with an HTML table of the top 10 via SMTP. If email fails, the report is still saved.	Email to the buyer + saved report

Technologies used

Layer	Technology	Used for
Agent orchestration	LangGraph	The agent graph: parallel scraping per site (Send API), parallel analysis agents, and rank waiting for all three
LLM	OpenAI API (Responses API)	All LLM agents. Structured outputs with Pydantic models, so every agent returns validated data. Model set in `.env` (gpt-4o)
Scraping	Tavily (Extract + Search)	Fetches the sites' search-results pages as markdown (advanced extract); web search as a fallback for other sites
Backend API	FastAPI + Uvicorn	REST endpoints: create, list, pause and delete searches, run now, report history
Scheduling	APScheduler	One daily job per saved search at the user's chosen time (Asia/Colombo time zone)
Database	SQLite + SQLAlchemy	Saved searches and the report from every run
Validation & config	Pydantic, pydantic-settings	API request checks and loading secrets from the `.env` file
Email	SMTP (Gmail App Password)	Sends the daily report; Python's built-in smtplib
Frontend	React + Vite	Form to create a search (vehicle, email, sites, time) and cards showing the latest top 10

Keeping LLM cost down

- Scraping, cleaning, filtering and de-duplication are plain Python; the LLM only sees real, matching cars.
- The sites do the filtering themselves (make, model, years, budget, gear, fuel) through their own search URLs.
- Each car is sent as a short record (about 10 fields), not the whole web page. In a test this cut each big prompt from about 15,000–24,000 characters to about 1,000–6,000.
- At most 25 cars are analysed per run, and cars without a seller description skip the condition LLM call.

Configuration

All secrets and settings live in `backend/.env`: OpenAI key and model, Tavily key, Gmail SMTP login (App Password), time zone, pages per site and the analysis cap. To add a site, write a URL builder in `agents/sites.py` and a card parser in `agents/sanitize.py`; to add an agent, add a node file in `agents/` and wire it in `graph.py`.